Open-Source CubeSat Optical Downlink Receiver and Processing System

Joshua Lazaruk[1], Erik Halliwell[1], Shayan Khorassany[1], K. Steven Knudsen[1]

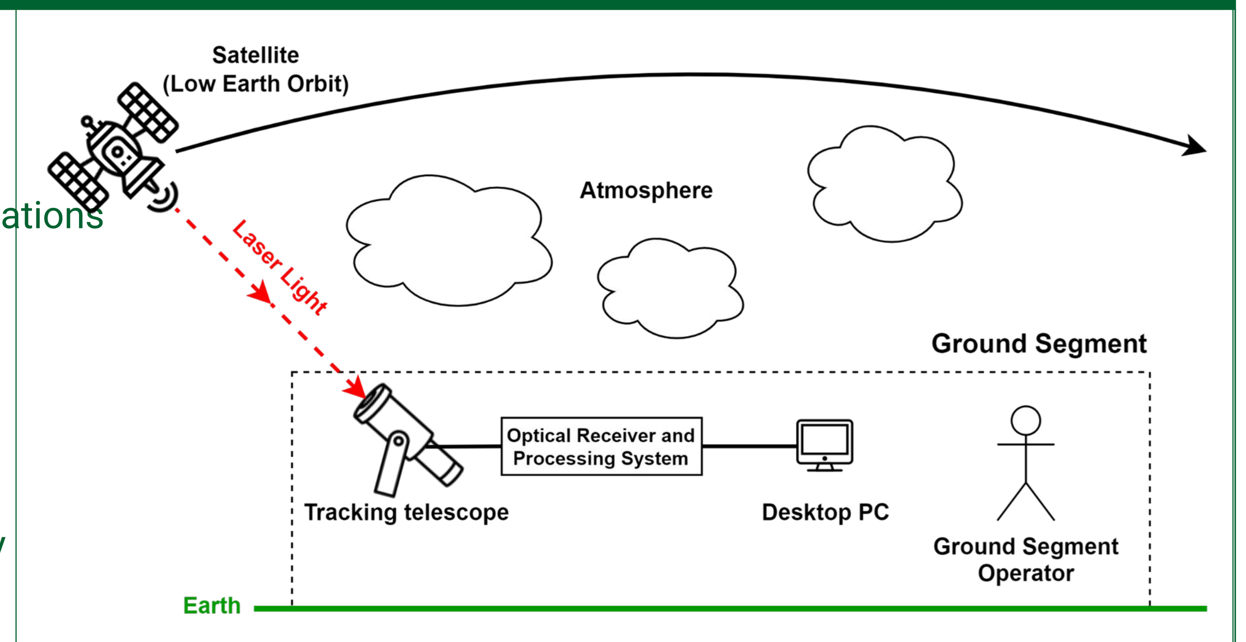
[1]Department of Electrical and Computer Engineering, University of Alberta, Edmonton, Canada

jdlazaru@ualberta.ca, ehalliwe@ualberta.ca, khorassa@ualberta.ca, knud@ualberta.ca

Motivation, Overview, and Requirements

Motivation

- Free-Space Optical Communication (FSO) for space applications is a promising alternative to radio communications
  - Not bandwidth limited
  - More power efficient
  - Less interference from other satellites
  - Eliminates regulatory/licensing efforts
- Widespread adoption is hindered by large barriers to entry



High-level system diagram

Overview

- An **open-source** optical signal processing system capable of receiving **on-off keying (OOK)** FSO has been created.
- The system will perform up to **25 Mbps**, with the ability to perform up to **100+ Mbps**
- A channel model simulation was created to analyze the performance of the receiver under realistic conditions.
- PHY/MAC layer includes **forward error correction (FEC)**, interleaving, and cyclic redundancy checks.
- Symbol (bit) and frame synchronization are implemented using **maximum length sequences (MLS)** and **polyphase autocorrelation**
- The system shall be able to be built by university cubesat groups.

Key Design Aspects

Empirical Channel Model

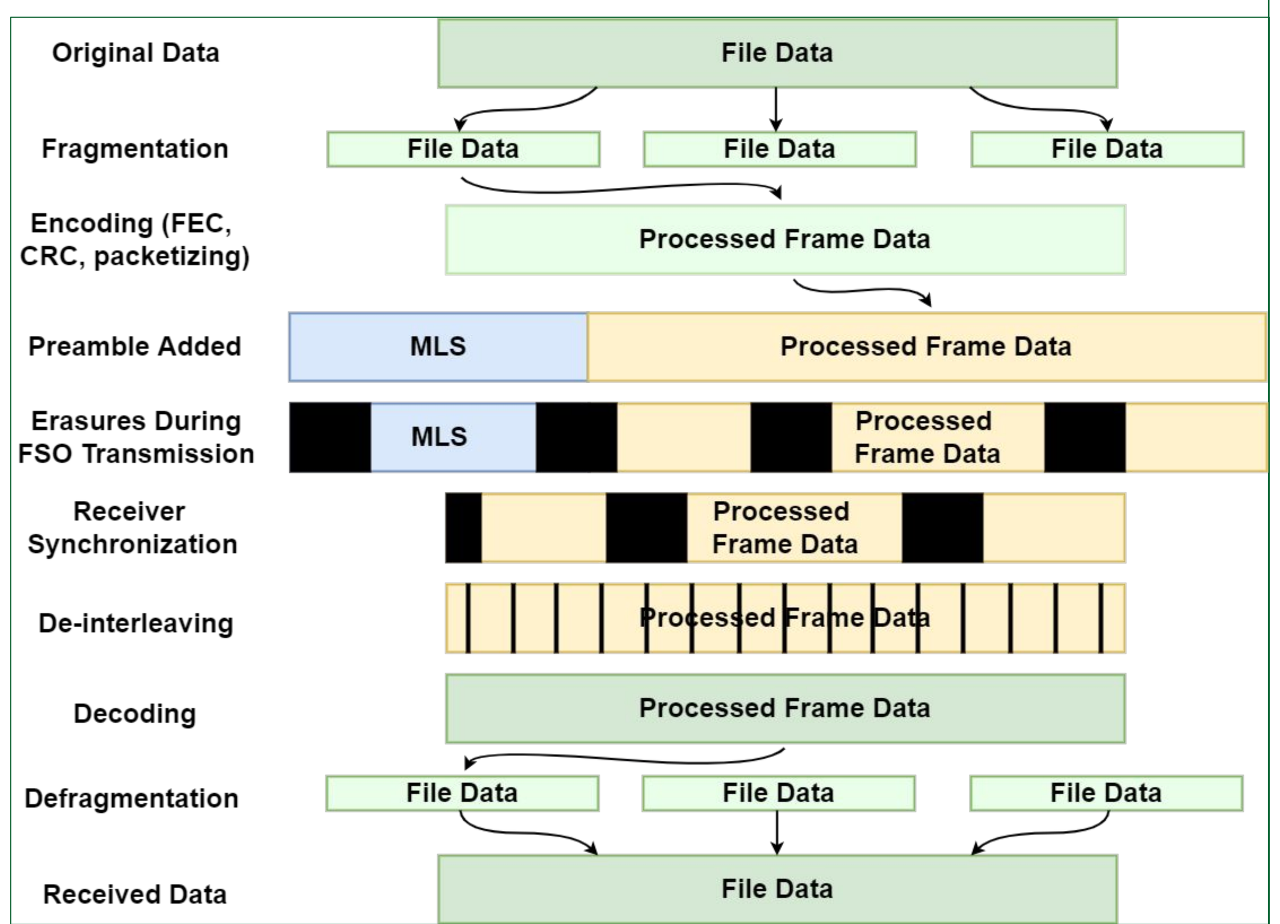
- Burst erasures (a result of atmospheric lensing) are the most prominent [1].
  - Simulated using Markov model
- Fade erasures become problematic at lower elevation angles [2, 3].
- White Gaussian noise is also added.

Forward Error Correction

- Low-density parity check (LDPC) FEC with interleaving can fully correct data that has been up to 80% erased.

Frame and Phase Synchronization

- Oscillator drift and doppler create loss of sync, limiting frame length
  - 18072 bits @ 0 degree elevation
  - 63274 bits @ 75 degree elevation
- MLS preamble is inherently resilient to erasures
- Preamble is autocorrelated in polyphase fashion on receiver every frame to maintain sync

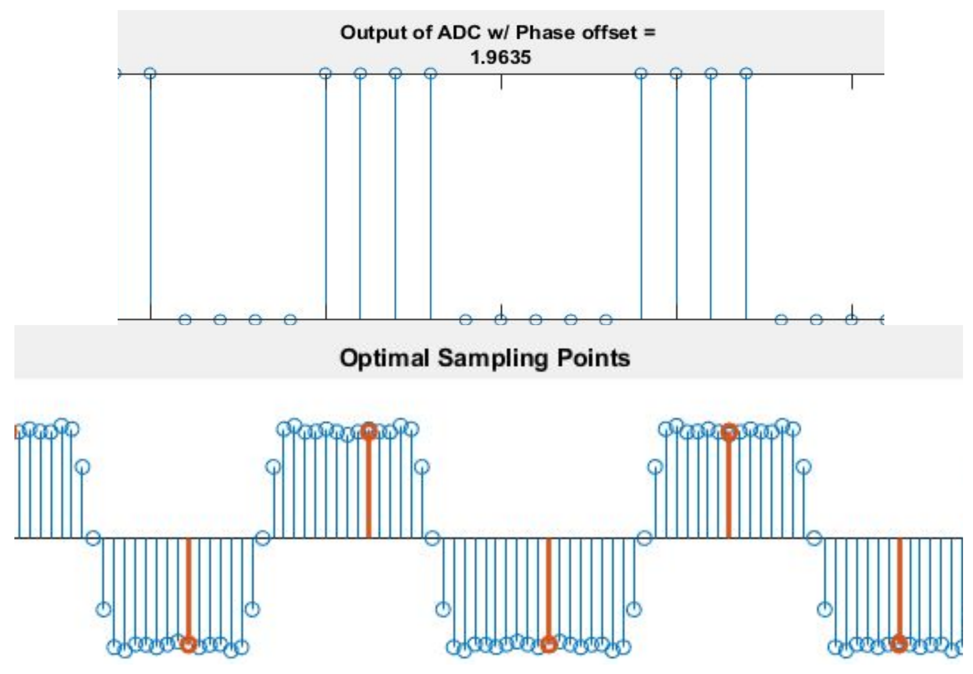


File transmit and receive process

Implementation

Simulation

- Transmitter, receiver, and channel software was developed in C/C++ using open-source signal processing and WiFi LDPC codes [4, 5].
- Current simulations predict a user data throughput of 5.77 Mbps (23%) at 25 Mbps.

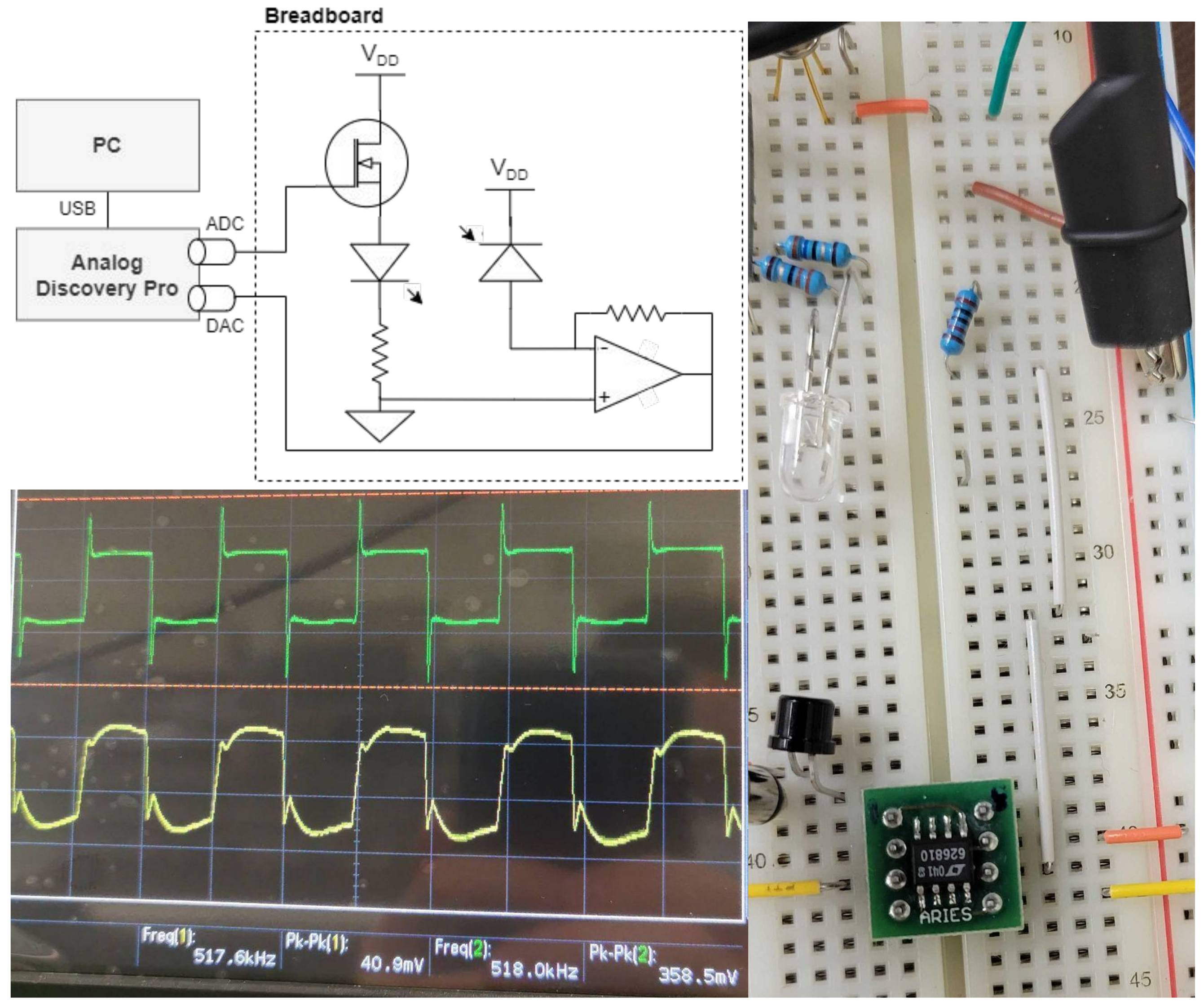


Top: analog-to-digital converter output

Bottom: upsampled signal, orange points are samples selected by sync algorithm

Hardware

- A 1+ Mbps proof-of-concept breadboard system has been demonstrated using an LED and photodiode.



Top-left: simplified schematic of hardware demo

Right: photo of breadboard hardware

Bottom-left: 1 Mbps OOK signal input to LED (green), and signal output by photodiode (yellow)

Summary of Results

- Fully modular, simulated system developed and ready to deploy on embedded hardware.
- Channel model emulates burst erasures and fades based on empirical data in addition to AWGN.
- Polyphase autocorrelator successfully synchronizes and demodulates received frame samples with realistic SNR, noise, and erasures present.
- LDPC FEC performs encoding by back-substitution and decoding using Belief Propagation.
- Open source: https://github.com/joshdellaz/laser_satellite_receiver

```
Original Data:
1031981051158125574236412051861712422512277012419484248272322311411
89046995115920115410250131834988163903793523882339421217117820519815
518084171413011665336122013511223362161652252521036

Prior to transmission:
128,185,160,252,72,22,126,97,244,253,11,123,19,194,129,212,63,131,24
8,182,209,140,174,146,9,58,242,198,30,192,173,142,4,93,75,58,164,52,
127,121,236,131,253,106,226,250,89,128,36,228,228,49,139,101,131,65,
89,213,147,25,248,21,43,220,176,6,254,96,65,49,159,132,88,151,64,100
,198,63,106,231,74,40,245,88,240,62,230,47,45,113,140,68,115,105,224
,58,94,247,80,103,203,49,165,31,208,220,8,159,242,40,113,145,176,140
,234,78,172,53,75,87,232,202,160,105,36,93,16,192,196,61,156,56,234,
202,239,20,154,146,149,90,127,106,16,151,193,19,95,43,41,162,208,70,
193,91,166,108,113,212,165,140,144,11,

Received Data:
128,185,160,252,72,22,126,97,244,253,11,123,19,194,128,_,_,_,_,_,1,1
40,174,146,9,58,242,198,30,192,173,142,4,93,75,58,164,52,127,121,236
,131,253,106,226,250,89,128,36,228,228,49,139,101,131,65,89,213,147,
153,248,21,43,220,176,6,254,96,65,49,159,132,88,151,64,100,198,63,10
6,231,74,40,245,88,240,62,230,47,45,113,140,68,115,105,224,58,94,247
,80,103,203,49,165,31,208,220,8,159,242,40,113,145,176,140,234,78,17
2,53,75,87,232,202,160,105,36,93,16,192,196,61,156,56,234,203,#,#,#,
#,#,#,#,#,#,#,#,#,#,#,#,#,#,#,#,#,230,108,113,212,165,140,144,11,

Decoded Data:
1031981051158125574236412051861712422512277012419484248272322311411
89046995115920115410250131834988163903793523882339421217117820519815
518084171413011665336122013511223362161652252521036

CRC Matches!
```

Data going through simulated system. "_" and "# " represent burst and fade erasures, respectively. Erased data is recovered by LDPC.

Future Work

End-to-End Simulation

- Full performance characterization and optimization (including performance at >25 Mbps)
- Modelling of fades and AWGN based on elevation angle [3]
- Adjustable packet and MLS length based on elevation angle to maximize throughput
- Use erasure location in FEC decoding
- Automatic gain control

Above and Beyond

- Design and build:
  - Cubesat transmitter subsystem
  - Receiver telescope
  - Tracking system

References

[1] Y. Yamashita, E. Okamoto, Y. Iwanami, Y. Shoji, M. Toyoshima and Y. Takayama, "An Efficient LDGM Coding Scheme for Optical Satellite-to-Ground Link Based on a New Channel Model," *2010 IEEE Global Telecommunications Conference GLOBECOM 2010*, 2010, pp. 1-6, doi: 10.1109/GLOCOM.2010.5683891.

[2] *ERASURE CORRECTING CODES FOR USE IN NEAR EARTH AND DEEP-SPACE COMMUNICATIONS*, CCSDS 131.5-O-1, November 2014.

[3] F. Moll, "Experimental analysis of channel coherence time and fading behavior in the LEO-ground link," in *Proc. on Int. Conf. on Space Optical Systems and Applications (ICSOS)*, May 2014

[4] J. D. Gaeddert, (2022) liquid-dsp (Version 1.4.0) [Source code]. https://github.com/jgaeddert/liquid-dsp.

[5] S. Tavildar, (2016) LDPC [Source code]. https://github.com/tavildar/LDPC.